Mahidol University International College

EGCI 463: Pattern Recognition
Final Project
Gold trend prediction

Nazneen Khanmongkhol **6481267**

Jinnaphat Guntawang **6481161**

Sopon Pleangnoi **6480646**

Mahidol University International College

Trimester 1/ 2025

December 4 , 2025

ABSTRACT

This project aims to utilize pattern recognition techniques to predict next-day fold prices using financial indicators including USD Index, crude oil prices, stock indices (NASDAQ, DJI, S&P500), Bitcoin and copper. We used 3 machine learning models (Random Forest, LSTM, Autoencoder + MLP). This methodology is selected as it allows 3 types of ML. Traditional ML baseline, Neural Network deep learning sequential model, and deep feature extraction and transfer-learning style model

The dataset is from 2015-2025, netting 1800-2000 aligned daily observations after merging. Feature engineering included lag values, moving averages, volatility, and returns. Evaluation metrics used are MAE, RMSE, and R^2.

Results highlight that all models struggle with the volatile financial markets, with none achieving satisfactory accuracy.The Autoencoder+MLP achieved the lowest MAE, Random Forest produced the most stable predictions, and the LSTM exhibited difficulty learning long-term temporal structure. The project demonstrates the challenges of financial time-series forecasting and the importance of model comparison in pattern recognition.

INTRODUCTION

Gold is one of the world's most important financial commodities, a status it has held virtually since the dawn of civilization. As history has shown, gold is the global symbol of wealth and the standard of exchange. It has been used as a measure of nations and individuals. Thus, gold is tied to global financial stability as a hedge against inflation and uncertainty.

Despite that, predicting movement of gold prices is a significant challenge as the price of gold is not directly correlated with anything but a combination of many financial variables together. It includes market volatility, global macroeconomic effects such as interest rate or country and central bank policies, combined with other non-linear relationships with other variables. To capture these very intricate, inter-dependent relationships is the biggest challenge to developing a reliable predictor.

With the challenging nature of the gold market, the objective of this project is more than just to make a model using pattern-recognition techniques but rather to determine the ability of these models to predict and capture complex pattern from a time-series data of gold price prediction. Using several methods, the goal is to find the best and reliable method for predicting gold price movements.

DATA

The dataset includes several **financial indices** used as predictors for gold prices. **Gold prices** act as a core predictor, utilizing the previous day's gold close price to estimate the next day's price. The **USD** (Gold price in USD value) reflects the strong negative correlation between the US Dollar and gold: a rising USD generally leads to falling gold prices, and vice versa.

Several stock market indicators capture market sentiment and risk appetite, including **DJI** (Dow Jones Industrial Average) and **NASDAQ** (NASDAQ), both of which typically show a negative correlation with gold (Risk-on sentiment decreases gold's appeal). The **S&P 500** (S&P 500) is a market sentiment indicator as it is usually posed with a negative correlation with gold.

Copper price (Copper closing price) is heavily linked to economic activity as it's shown to fluctuate with inflation expectations and market demand while **Crude oil** (Crude oil closing price) reflects inflation and global economic health, usually showing a positive correlation where high oil prices drive gold prices up. Finally, **BTC** (Bitcoin closing price) is included to help capture movement in the risk sentiment and the dynamics of cryptocurrency and gold.

Data preparation

The initial data preparation process took several steps to make sure the time-series models use a clean and properly-scaled input. First we download the dataset from year 2015 to year 2025, the dataset is cleaned by deleting missing values to prevent errors during model training. Then, all of the timestamp columns are then converted to date as the project only aims to predict the daily movement and to simplify the data at hand. The next step is to match the data to the gold price update, as the gold prices are sourced from the future market and future gold prices only update on weekdays unlike other prices. This ensures that other features such as BTC, US, or Oil prices align with the gold prices available.

Finally, all of the numerical features are standardized using the MinMaxScaler, transforming the value to the suitable range as values of each index/commodity are not on the scale. This is essential for a stable model convergence and improves the maximal capacity of performance of each model.

After the cleaning processes, the merging process is done on the common date column to create a single, unified dataset. This combined DataFrame includes the closing values for all the financial features being used as predictors: Gold close (close), Oil close (close_oil), USD close (close_usd), DJI close (close_dji), Copper close (close_cop), NASDAQ close (close_nas), S&P close (close_sp), and BTC close (close_btc).

```python
# Convert to datetime
gold['datetime'] = pd.to_datetime(gold['datetime'])
oil['datetime']  = pd.to_datetime(oil['datetime'])
usd['datetime']  = pd.to_datetime(usd['datetime'])
dji['datetime']  = pd.to_datetime(dji['datetime'])
cop['datetime']  = pd.to_datetime(cop['datetime'])
nas['datetime']  = pd.to_datetime(nas['datetime'])
sp['datetime']  = pd.to_datetime(sp['datetime'])
btc['datetime']  = pd.to_datetime(btc['datetime'])
```

```python
# Gold goes back to 1800 (Keep only matching years)
gold = gold[gold['datetime'] >= "2015-01-01"]
```

```python
# Convert timestamp → date only
gold['date'] = gold['datetime'].dt.date
oil['date']  = oil['datetime'].dt.date
usd['date']  = usd['datetime'].dt.date
dji['date']  = dji['datetime'].dt.date
cop['date']  = cop['datetime'].dt.date
nas['date']  = nas['datetime'].dt.date
sp['date']  = sp['datetime'].dt.date
btc['date']  = btc['datetime'].dt.date
```

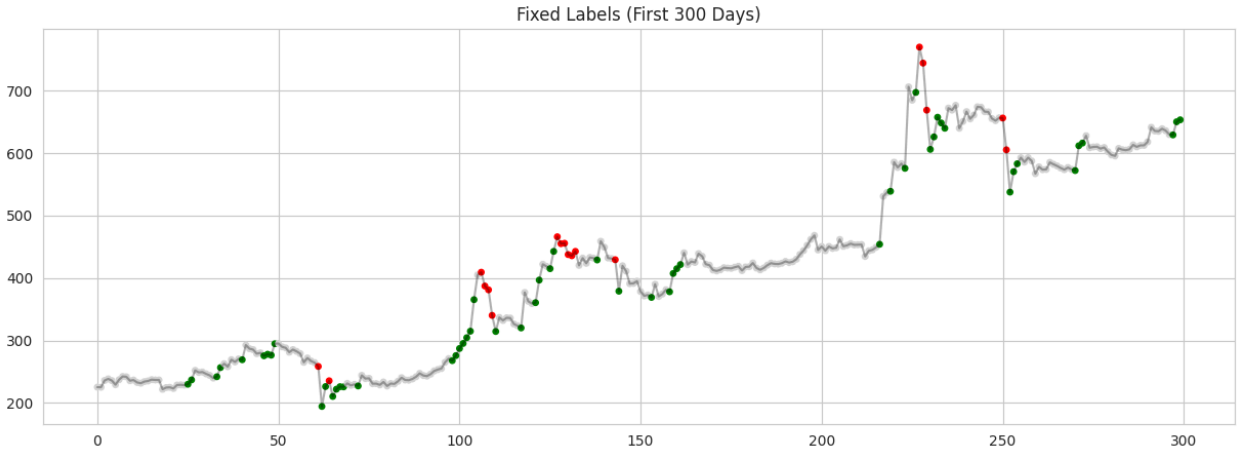
```python
# Merge all

dfs_to_merge = {
    "gold": gold,
    "usd": usd,
    "dji": dji,
    "cop": cop,
    "nas": nas,
    "sp": sp,
    "oil": oil,
    "btc": btc
}

df = gold.copy()

for name, current_df in dfs_to_merge.items():
    if not current_df.empty:
        if name != "gold": # gold is the base
            df = df.merge(current_df, on="date", how="inner", suffixes=("", f"_{name}"))
    else:
        print(f"Warning: '{name}' dataframe is empty and will not be merged.")
```

```python
# 3. Scale Features
scaler_x = MinMaxScaler()
X_scaled = pd.DataFrame(scaler_x.fit_transform(X), columns=X.columns)
```

Data preparation (2)

After we are done with adding all the clean columns and rows together, we now move on to the most important part of the models. The output, as we stand, is just a spreadsheet of numbers with date and label attached to it. To accurately predict the direction of gold price, the model must have a reliable PIP(price index points) as a basis to work on.

First, we used log transformation on the data then we applied standard PIP logic onto it so that we have the key turning points. This repeats until the maximum distance of the point is less than 5% then we convert the data back so that it is easier to read. Then we used an auto calculated trend label function so that each slope is calculated by percentage changes to act as the direction finders rather than using fixed numbers since gold prices fluctuate massively. Using this method we can determine the flattest part of the gold price graph as the threshold for the data to act as a neutral which was deemed to be around 33% for the data to have as much balance as possible. Our labels are 0 (down), 1 (up), and 2 (neutral).



From this figure, when gold is having a down trend, it shows red. When it is having an up trend, it shows green. And if the trend is not dramatic, it shows gray as neutral.

```python
def find_pips_log_safe(data_values, dist_threshold=0.05):
    """
    Finds PIPs using LOG scale to handle 2015 vs 2025 price differences.
    """
    # 1. Log Transform safely
    log_data = np.log(data_values)

    # 2. Standard PIPs logic on the LOG data
    pips_x = [0, len(log_data) - 1]
    pips_y_log = [log_data[0], log_data[-1]]

    while True:
        max_dist = 0.0
        best_idx = -1
        insert_pos = -1

        for k in range(len(pips_x) - 1):
            idx_left = pips_x[k]
            idx_right = pips_x[k+1]

            if idx_right - idx_left <= 1: continue

            # Equation for line between two log points
            slope = (pips_y_log[k+1] - pips_y_log[k]) / (idx_right - idx_left)
            intercept = pips_y_log[k] - slope * idx_left

            # Check all points in between
            for i in range(idx_left + 1, idx_right):
                # Vertical distance (Perpendicular distance logic simplified for speed)
                # d = |Ax + By + C| / sqrt(A^2 + B^2). Here y = slope*x + intercept.
                est_y = slope * i + intercept
                d = abs(log_data[i] - est_y) # Vertical distance in Log Space

                if d > max_dist:
                    max_dist = d
                    best_idx = i
                    insert_pos = k + 1

        if max_dist < dist_threshold or best_idx == -1:
            break

        pips_x.insert(insert_pos, best_idx)
        pips_y_log.insert(insert_pos, log_data[best_idx])

    # 3. Convert Log Y-values back to Real Dollars
    pips_y_real = np.exp(pips_y_log)
    return pips_x, pips_y_real
```

```python
def label_trends_auto(pips_x, pips_y, data_len, neutral_percentile=33):
    """
    Auto-calculates threshold so the flattest 33% of trends are Neutral.
    """
    slopes = []
    # 1. Calculate all slopes (Percentage Change per Day)
    for i in range(len(pips_x) - 1):
        dx = pips_x[i+1] - pips_x[i]
        if dx == 0: continue

        # % Change = (New - Old) / Old
        pct_change = (pips_y[i+1] - pips_y[i]) / pips_y[i]
        daily_slope = pct_change / dx
        slopes.append(daily_slope)

    # 2. Find the threshold that separates the bottom 33%
    abs_slopes = np.abs(slopes)
    dynamic_threshold = np.percentile(abs_slopes, neutral_percentile)

    print(f"Stats: Found {len(pips_x)} PIPs.")
    print(f"Stats: Auto-Threshold is {dynamic_threshold:.6f} (Daily % Change)")

    # 3. Apply Labels
    labels = np.zeros(data_len, dtype=int)
    for i in range(len(pips_x) - 1):
        x0, x1 = pips_x[i], pips_x[i+1]
        y0, y1 = pips_y[i], pips_y[i+1]

        if x1 - x0 == 0: continue

        daily_slope = ((y1 - y0) / y0) / (x1 - x0)

        if abs(daily_slope) < dynamic_threshold:
            l = 2 # Neutral
        elif daily_slope > 0:
            l = 1 # Up
        else:
            l = 0 # Down

        labels[x0:x1+1] = l

    return labels
```

FEATURE ENGINEERING

To extract meaningful patterns and enhance the predictive power of the models, several types of engineered features were created.

Pattern Extraction Features:

Lag Features: Three lag features were chosen to capture autoregressive behavior while avoiding capturing unwanted trends from other macro factors. These features represent the closing price from one day prior, one week prior, and two weeks prior. Lag features are essential for allowing the model to identify short-term momentum and price reversals.

Moving Averages (MAs): Three Moving Averages with lookback periods of 5, 20, and 50 days were calculated. These MAs serve to smooth out noise and represent distinct market trends: the 5-day MA for short-term trends, the 20-day MA for medium-term trends, and the 50-day MA for long-term trends. This helps the model detect underlying directional movement separate from the daily randomness accounted for by the lag features.

Volatility and Daily Return: Volatility was added, calculated as the 10-day standard deviation of prices, to show periods of risk and instability. This was paired with Daily Return, calculated as the percentage change from the previous day. This is crucial because periods of high volatility strongly influence future price movements.

```
# Create Lag Features
df['lag1'] = df['close'].shift(1)
df['lag5'] = df['close'].shift(5)
df['lag10'] = df['close'].shift(10)
```

```
# Moving Averages
df['MA5'] = df['close'].rolling(5).mean()
df['MA20'] = df['close'].rolling(20).mean()
df['MA50'] = df['close'].rolling(50).mean()
```

```
# Volatility + Returns
df['volatility'] = df['close'].rolling(window=10).std()
df['return'] = df['close'].pct_change()
```

METHODS

1. Random Forest Regressor (Traditional ML)

The first architecture we chose was Random Forest as it is a traditional model that we can hand-pick the features for. It is also known to be able to handle non linear relationships and its a fast and stable baseline.

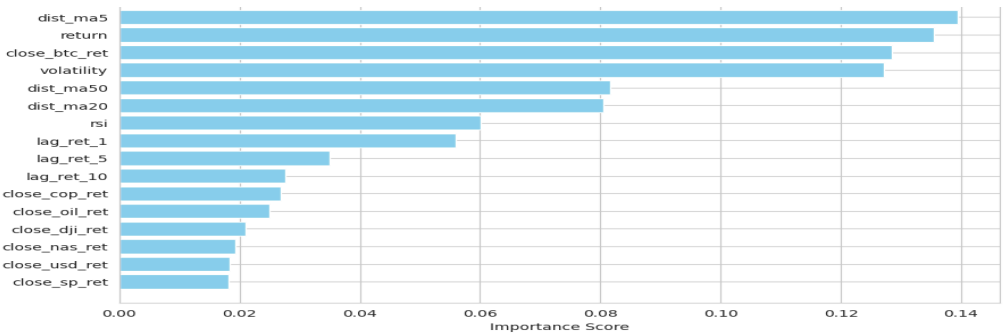
We used a parameter gird into RandomSearchCV to find the best parameters for the models as it is the most straightforward model in the bunch. And our list of parameters are n_estimators, max_depth, min_sample_spilt, min_sample_leaves, and max features. We then train with other fixed parameters as the standard.

```
param_dist = {
    'n_estimators': [300, 500, 800],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}
```

```
# 3. Grid Search with classification scoring
random_search = RandomizedSearchCV(
    rf,
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring='f1',
    verbose=1,
    n_jobs=-1
)
```

However, even with this the model was lacking, so what we did was to implement the model with hand picked features using a importance from the library as the determiner.

Using that code block, we found that the most important features are 'dist_ma5', 'return', 'close_btc_ret', 'volatility', 'dist_ma50', 'dist_ma20', 'rsi', 'lag_ret_1'. The key observation we can see is that the other indices were not of importance to the model at all. With most having only around 0.2 importance.



2. LSTM Regression (Deep Learning)

LSTM is designed for sequential data, capturing temporal dependencies that traditional ML models cannot. This model used the pruned features that we acquired from the RF method. As we aimed to bring out the best performance possible. Then we use sequence generation which is 14 days as the longer lookback window was selected to gather immediate momentum and volatility, as when we tried with the longer 60 days sequence, it was not optimal.

Then we used oversampling to compensate for the unbalanced dataset as most of the gold prices direction is neutral. This way we can force feeding the model and minimizing loss as the model wont only pick the most found class (neutral)The model is a bidirectional LSTM with 2 layers structure of 64 and 32 then the output is 3 for the 3 classes.

We also implemented EarlyStopping callback with patience of 8 so that it dynamically determines the optimal number of epochs with val_loss to prevent the model from memorizing noise. And then we determined the confidence to find the best threshold to manually force the model to perform as the base behavior was very shy and it mostly pick neutral to be safe

```python
# --- 1. DEFINE OVERSAMPLING FUNCTION ---
def oversample_minority_classes(X, y):
    # Identify indices for each class
    idx_0 = np.where(y == 0)[0] # Down
    idx_1 = np.where(y == 1)[0] # Up
    idx_2 = np.where(y == 2)[0] # Neutral

    # Calculate how many we need to match the majority class (Neutral)
    max_count = len(idx_2)

    # Randomly sample (duplicate) the minority classes to match the majority
    # replace=True allows duplicating the same sample multiple times
    idx_0_up = np.random.choice(idx_0, size=max_count, replace=True)
    idx_1_up = np.random.choice(idx_1, size=max_count, replace=True)

    # Combine all indices
    all_indices = np.concatenate([idx_0_up, idx_1_up, idx_2])

    # Shuffle so the model doesn't learn the order
    np.random.shuffle(all_indices)

    return X[all_indices], y[all_indices]
```

```python
# --- 4. BUILD MODEL (Same Architecture) ---
model = Sequential([
    Input(shape=(SEQ_LEN, len(pruned_features))), # Input features = 8 now

    Bidirectional(LSTM(64, return_sequences=True)),
    Dropout(0.3),

    Bidirectional(LSTM(32, return_sequences=False)),
    Dropout(0.3),

    Dense(16, activation='relu'),
    Dense(3, activation='softmax')
])
```

```python
model.compile(optimizer=Adam(learning_rate=0.001), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_loss', patience=8, restore_best_weights=True)
```

3. Autoencoder + MLP (Transfer Learning Style)

Autoencoders compress high-dimensional data into latent features, learning nonlinear structure without supervision. We start by flattening the data with the sequence with the oversampling the same way as the LSTM model. This model instead uses 112 features with 2 layers of 64 and 32 with the latent code layer of 8 in the end. We then used the output to do 2 things, reconstruction and classification to denoising and capture the structural features of the price movement. And to train the latent code to predict the trend direction as a transfer learning component.

```python
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam

# 1. FLATTEN THE DATA
input_dim = X_train_balanced.shape[1] * X_train_balanced.shape[2] # 14 * 8 = 112

X_train_flat = X_train_balanced.reshape(X_train_balanced.shape[0], input_dim)
X_test_flat  = X_seq_test.reshape(X_seq_test.shape[0], input_dim)

print(f"Flattened Input Shape: {X_train_flat.shape}")
# Expected: (Samples, 112)

# 2. DEFINE ENCODER
encoding_dim = 8

input_layer = Input(shape=(input_dim,))

# Compressing
encoded = Dense(64, activation='relu')(input_layer)
encoded = Dense(32, activation='relu')(encoded)
encoded_output = Dense(encoding_dim, activation='relu')(encoded) # Latent Features

# Decompressing (Reconstruction)
decoded = Dense(32, activation='relu')(encoded_output)
decoded = Dense(64, activation='relu')(decoded)
decoded_output = Dense(input_dim, activation='linear')(decoded)

# 3. BUILD & TRAIN
autoencoder = Model(input_layer, decoded_output)
encoder = Model(input_layer, encoded_output)

autoencoder.compile(optimizer='adam', loss='mse')

print("Training Autoencoder...")
autoencoder.fit(
    X_train_flat, X_train_flat, # Input = Output (Unsupervised)
    epochs=100, # Reduced from 400 (sufficient for small data)
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
```

MLP Regressor

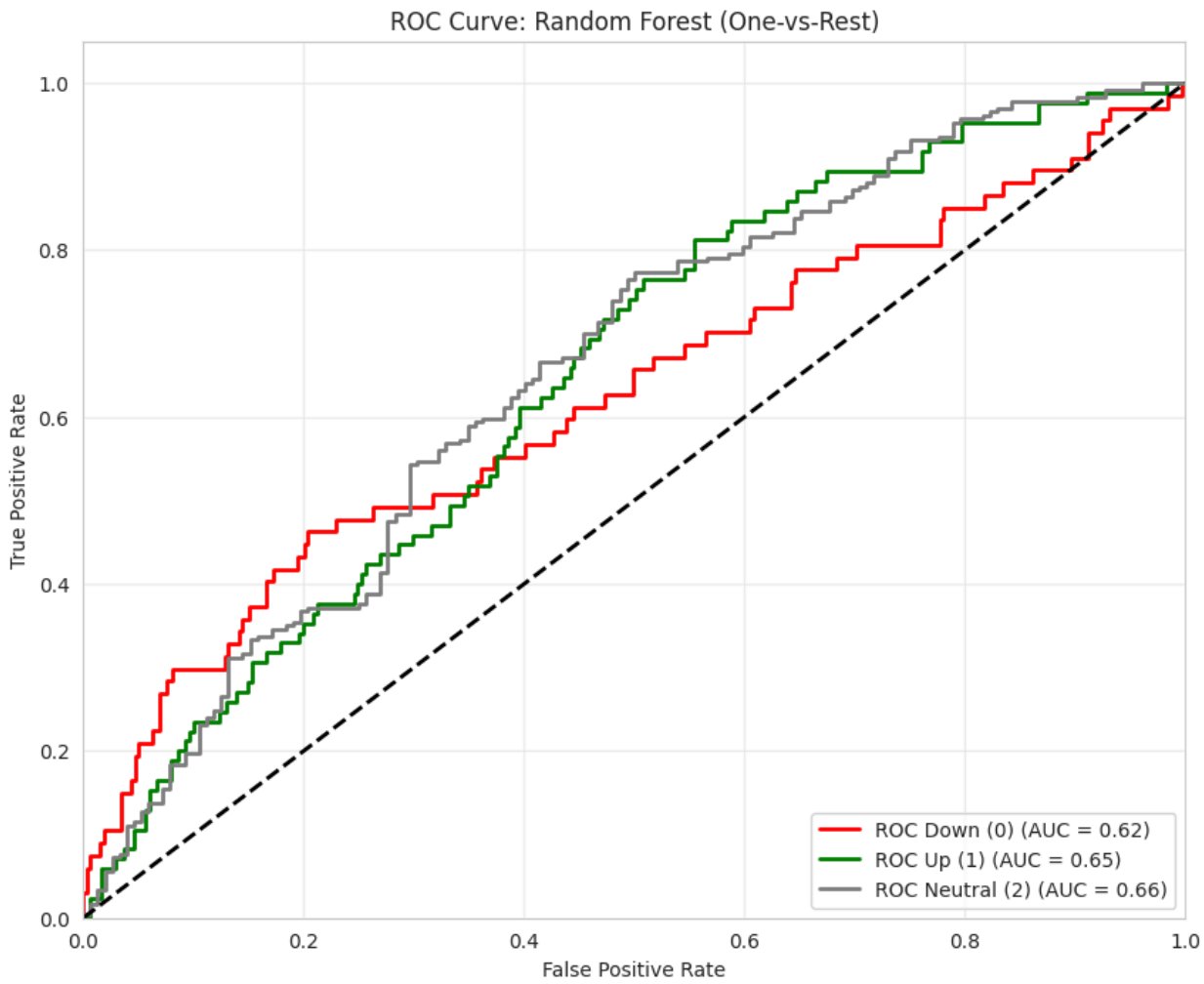
We use the previously trained encoder to process the 112 feature flattened data. We used the latent features into the new input for MLP. The architecture of MLP is 2 layers of 64 and 32 with 2 dropouts of 0.3 and 0.2 in between then we get the 3 neuron classifiers as the output.

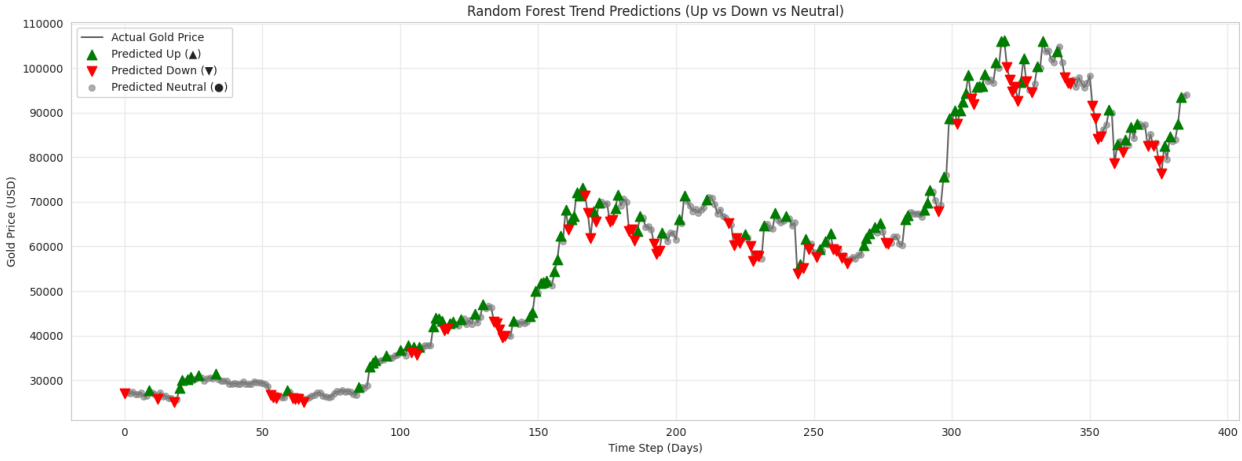
We used EarlyStopping as the callback monitors val_loss and stop train if there are no improvements (in this case 10 epoch) as it determines the optimal number. ReduceLROnPlateau is used for reducing the lr if the model stalls so that it can escape the local minimum and helps convergence

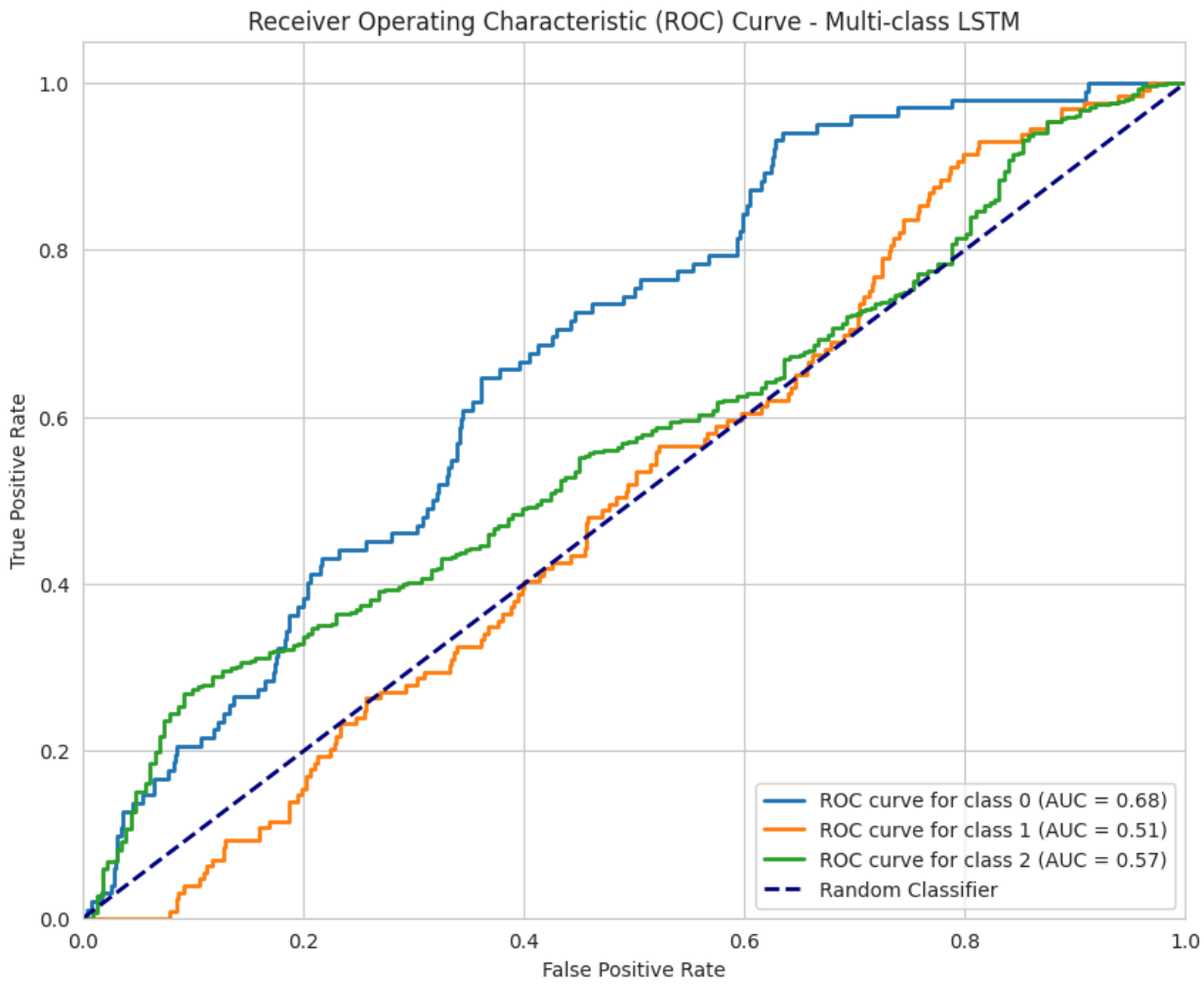
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.utils import to_categorical

# --- 1. COMPRESS THE DATA (Feature Extraction) ---

print("Compressing data...")
X_train_encoded = encoder.predict(X_train_flat)
X_test_encoded  = encoder.predict(X_test_flat)

print(f"Compressed Shape: {X_train_encoded.shape}")
# Expected: (Samples, 16)

# --- 2. BUILD THE CLASSIFIER (MLP) ---
# Adapted for 3 Classes (Down=0, Up=1, Neutral=2)
cls_mlp = Sequential([
    # Input is the 16 Latent Features
    Dense(64, activation='relu', input_shape=(encoding_dim,)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(32, activation='relu'),
    Dropout(0.2),

    # Output: 3 Neurons (one for each class), Softmax activation
    Dense(3, activation='softmax')
])

cls_mlp.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy', # Handles integers 0, 1, 2 directly
    metrics=['accuracy']
)

# --- 3. TRAIN ---
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)

print("Training MLP Classifier...")
history_mlp = cls_mlp.fit(
    X_train_encoded, y_train_balanced, # Train on Compressed Data vs Balanced Labels
    validation_data=(X_test_encoded, y_seq_test), # Validate on Real Data
    epochs=100,
    batch_size=64,
    callbacks=[early_stop, lr_scheduler],
    verbose=1
)
```

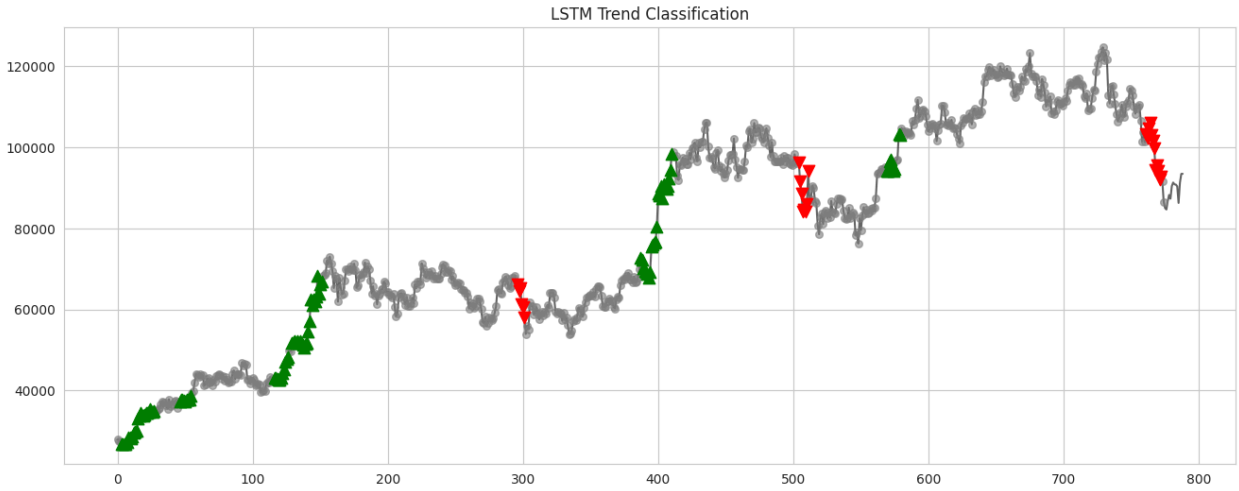
Results

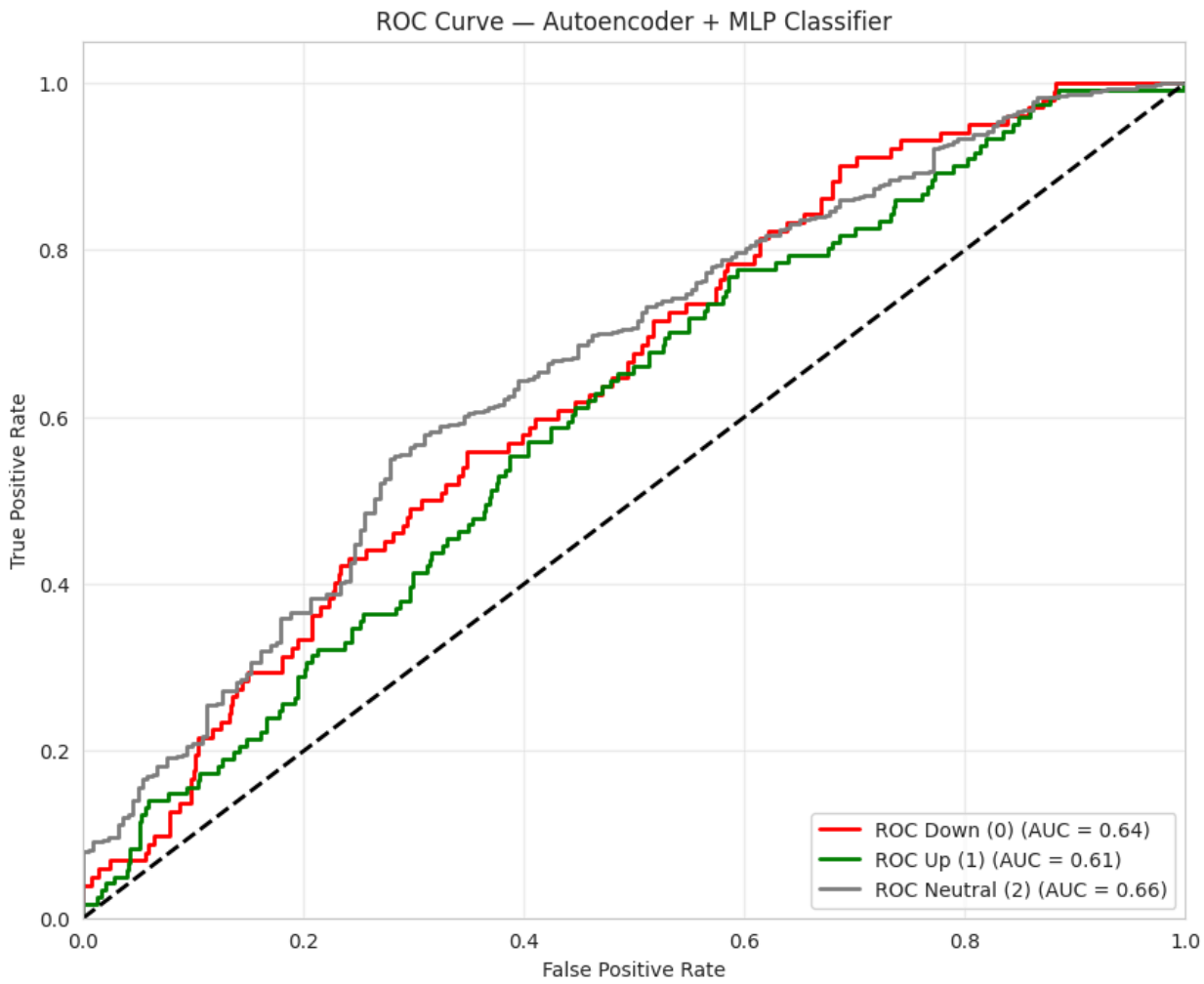
Model Architecture	Accuracy	Weighted F1	Up (1) F1	Down (0) F1	Neutral (2) F1	Key Characteristic / Failure Mode
Optimized Random Forest	**58.3%**	**0.57**	**0.32**	**0.34**	0.73	**Best Baseline.**
Autoencoder + MLP	63.0%	0.59	0.21	0.08	0.79	**Regime Specialist.**
Balanced LSTM	35.0%	0.39	0.26	0.18	0.46	**Deep Learning Failure.**
Supervised Autoencoder	70.0%	0.58	0.00	0.07	**0.82**	**Mode Collapse.**

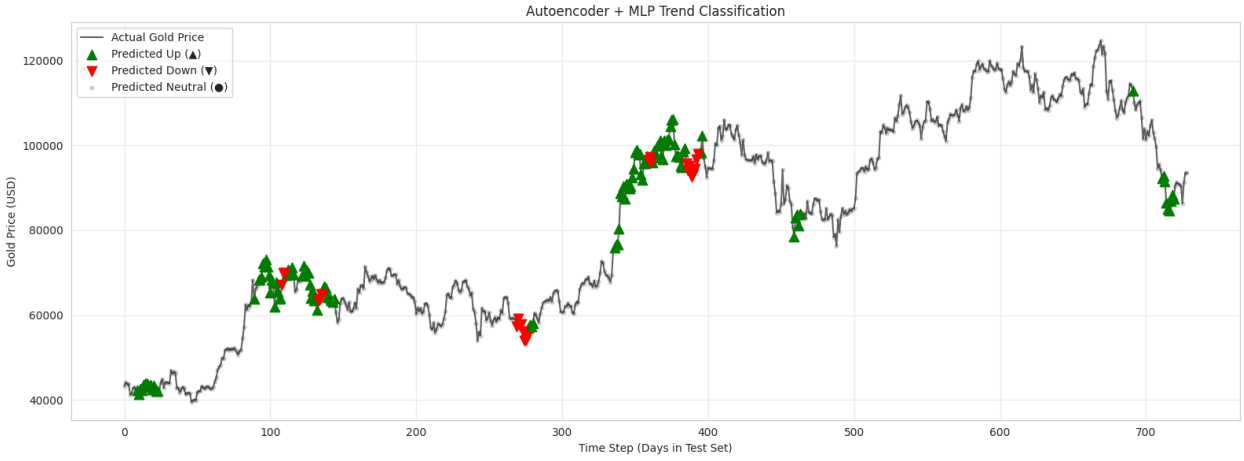

ROC Curve: Random Forest (One-vs-Rest)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC Down (0) (AUC = 0.62)
ROC Up (1) (AUC = 0.65)
ROC Neutral (2) (AUC = 0.66)


Random Forest Trend Predictions (Up vs Down vs Neutral)
Actual Gold Price
Predicted Up (▲)
Predicted Down (▼)
Predicted Neutral (●)
Gold Price (USD)
Time Step (Days)
30000
40000
50000
60000
70000
80000
90000
100000
110000
0
50
100
150
200
250
300
350
400


Receiver Operating Characteristic (ROC) Curve - Multi-class LSTM
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve for class 0 (AUC = 0.68)
ROC curve for class 1 (AUC = 0.51)
ROC curve for class 2 (AUC = 0.57)
Random Classifier


LSTM Trend Classification
120000
100000
80000
60000
40000
0
100
200
300
400
500
600
700
800


ROC Curve — Autoencoder + MLP Classifier
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC Down (0) (AUC = 0.64)
ROC Up (1) (AUC = 0.61)
ROC Neutral (2) (AUC = 0.66)


Autoencoder + MLP Trend Classification
Actual Gold Price
Predicted Up (▲)
Predicted Down (▼)
Predicted Neutral (●)
Gold Price (USD)
Time Step (Days in Test Set)
40000
60000
80000
100000
120000
0
100
200
300
400
500
600
700

DISCUSSION

Our project shows that ensemble methods are significantly better for this task as a daily trend classification model. The optimized Random Forest was the model with highest accuracy with most balance on all sides. On the other hand, the Deep learning model is much worse accuracy wise at around 35% as it defaults to neutral or class 2 since the data is quite messy. Autocoder + MLP however got a much better score but only works with some dimensions as the score highly relies on the fact that the data is mostly neutral.

But that is not all of the truth as the score only shows how much of the prediction was correct but from the plotted graph and underlying metrics, the best model is by far the Random Forest as it give outs the most predictions with the most accurate results this presents the idea that the predictive signal is linear and feature based not the dependent on temporal sequences as the LSTM only “catch the trends” as it predicted neutral almost exclusively unless there are big shift. And the Autoencode + MLP has similar behavior with more accuracy but is much more reluctant to predict the direction. Thus, Deep learning models are more suitable as the volatility filters rather than prediction models.